

1. Delaunay-Triangulation

Grundfrage: Wie kommen wir von einer Menge an Vertices zu einer Menge an sinnvollen Dreiecken? → Triangulierungs-Algorithmen

1.1. Schritte des Bowyer-Watson Algorithmus

1. Erschaffe ein *Super-Triangle*, welches alle Punkte umschließt
2. Füge einen neuen Punkt hinzu
3. Finde alle invaliden Dreiecke, deren Umkreis den neuen Punkt umschließen
4. Erstelle ein Polygon aus allen invaliden Dreiecken
5. Verbinde die Punkte des Polygons mit dem neuen Punkt, um neue Dreiecke zu erschaffen
6. Entferne die invaliden Dreiecke
7. Wiederhole ab 2. bis alle Punkte eingefügt wurden
8. Entferne alle Dreiecke, die noch mit dem *Super-Triangle* verwandt sind

Aufgabe: Implementiere die Delaunay-Triangulation unter der Nutzung des Bowyer-Watson Algorithmus (Falls andere Ansätze genutzt werden wollen, ist das auch möglich)

1.2. Hilfreiche Links

- https://en.wikipedia.org/wiki/Delaunay_triangulation
- https://en.wikipedia.org/wiki/Bowyer%E2%80%93Watson_algorithm
- <https://www.gorillasun.de/blog/bowyer-watson-algorithm-for-delaunay-triangulation/>

1.3. Java Projekt

In dem Ordner `OpenGL/lwjgl-project` und in dem Package `java.com.xpromus.tasks` befindet sich das `triangulation` Package. In diesem Package befinden sich folgende Dateien:

- `TaskMain.java`: Sie führt die Übung mit einer Menge an Punkten, unter der Nutzung von OpenGL aus und sollte bei korrekter Implementierung eine Mesh anzeigen
- `Triangulator.java`: Sie enthält alle Funktionen für die Triangulierung. Die Implementierung wird hier im Laufe der Übung geschehen

Boilerplate-Code und Datentypen wurden bereits implementiert. Auch hilfreiche Funktionen, wie zum Beispiel zur Berechnung des Umkreises eines Dreiecks, werden bereitgestellt (zu finden in den Klassen im `data` Package) Der Fokus liegt hier nur auf der Logik des Algorithmus.

1.4. Zu erwartendes Ergebnis

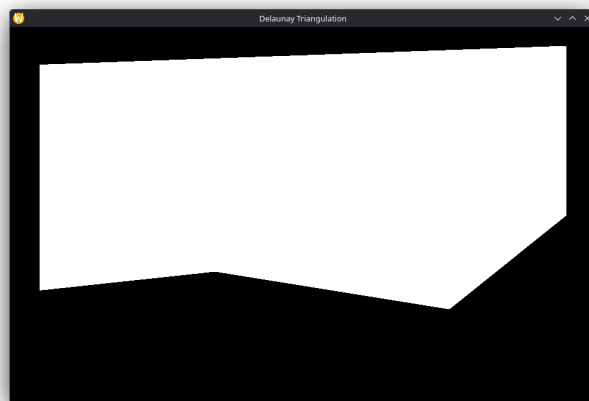


Figure 1: Ergebnis bei den gegebenen Beispielpunkten